

oslab-project3

Design review

? Question

请展示spawn、kill、wait和exit的伪代码

```
sys_exec(name) -> pid_t:
    pcb = alloc_pcb()
    load_program(pcb, name)
    init_context(pcb)
    pcb.status = READY
    push_to_ready_queue(pcb)
    return pcb.pid

sys_kill(pid):
    pcb = find_pcb(pid)
    if pcb is NULL: return -1
    release_all_locks(pcb)      // 若它持有锁必须释放
    wakeup_all_waiters(pid)
    free_pcb_and_stack(pcb)
    if pcb == current_running:
        schedule()
    else:
        remove_from_ready_or_block_queue(pcb)
    return 0

sys_exit() -> NORETURN:
    cur = current_running
    release_all_locks(cur)
    wakeup_all_waiters(cur.pid)
    free_pcb_and_stack(cur)
    schedule()    // never return

sys_waitpid(pid) -> int:
    if target process has finished:
        return target.exit_code
    add current_running to wait_queue[pid]
    block(current_running)
    schedule()
    return target.exit_code
```

? Question

当kill一个持有锁的进程时，kill的实现中要进行哪些处理？

1. 释放它持有的所有锁
2. 清理它在所有同步结构中的痕迹
 - 从 semaphore/block/condvar 等等待队列删除它

- 从 ready queue / sleep queue 中删除

3. 唤醒所有 wait(pid) 的进程

4. 回收 PCB 与 栈空间

🔗 Question

如何实现条件变量、屏障？请简述你的设计思路或展示伪代码。在使用条件变量、屏障时，如果有定时器中断发生，你的内核会做什么处理么？

```
struct Cond {
    void wait(mutex_t mutex);
    void signal();
    void broadcast();
private:
    Queue<pid_t> wait_queue;
};

void Cond::wait(mutex) {
    release(mutex)
    this->wait_queue.add(current_running)
    block(current_running)
    schedule()
    acquire(mutex)
}

void Cond::signal() {
    if (!this->wait_queue.empty()) {
        wake_up_one(this->wait_queue);
    }
}

void Cond::broadcast() {
    wake_up_all(this->wait_queue)
}
```

```
struct Barrier {
    Barrier(int total) : total(total), count(0) {}
    void wait();
private:
    int total, count;
};

void Barrier::wait() {
    bar.count++
    if (bar.count < bar.total) {
        block(current_running)
        add to bar.wait_queue
        schedule()
    } else {
        bar.count = 0
        wake_up_all(bar.wait_queue)
    }
}
```

定时器中断不会影响同步原语，因为同步原语执行的时候肯定是内核态，进入内核态的时候中断使能已经关了

🔍 Question

简述如何保护mailbox并发访问的正确性？

mutex 保护自身数据结构，然后用两个 cond 或者一个 semaphore 控制缓冲区读写

```
mailbox.put(self, item):
    self.lock.acquire()
    while self.full:
        self.not_full.wait(self.lock)
    enqueue(item)
    self.not_empty.signal()
    self.lock.release()

mailbox.get(self):
    acquire(self.lock)
    while self.empty:
        self.not_empty.wait(self.lock)
    item = dequeue()
    self.not_full.signal()
    self.lock.release()
    return item
```

🔍 Question

如何让主核（core 0）和从核（core 1）正常工作？

主核做完初始化之后通知从核

共享数据结构用锁保护

在 `ret_from_exception` 里面 `unlock_kernel`，然后在 `exception_handler_entry` 里面 `lock_kernel`

init 的 pcb 比较奇怪，现在是这么设计的：用一个保证不会用到 `$sp` 的函数作为虚拟 task 的 entrypoint，然后调度的时候 `sret` 跳到这个 entrypoint，同时 `sp` 转成 `user_sp`，其中因为不需要用到 `sp`，可以不分配 user 栈空间

```
static void spin() {
    // NOTE: this function should NOT use stack pointer (because user_sp is 0 for kernel
    pcb)
    while (true) {
        // enable_preempt(); NOTE: this code is runned in U-mode
        asm volatile("wfi");
    }
}

/* ... */
```

bug: `time_base` 被初始化之后重新设置成 0，问题： `_start` 中的初始化 `bss` 被执行了两次

bug: 自旋锁实现有问题，框架里面给的 `atomic_cmpxchg(old, new, addr)` 逻辑是：

- `*addr == old` `*addr = new, return old`

- `*addr != old` `return *addr`

```
retry:
    ret = *mem_addr
    if (ret != old_val)
        goto done
    if (SC(mem_addr, new_val) failed)
        goto retry
    fence    // 成功路径保证语义
done:
    return ret
```